

Codex CLI 一页黑白海报

给终端工作流用的最短心智模型：界面怎么显示、旧对话怎么找、项目里该怎么管线程。

codex	临时全屏覆盖当前终端
cx	<code>codex --no-alt-screen</code> ，直接接在当前终端历史下面继续输出
resume	回到旧的那条对话继续
fork	从旧对话分叉出一条新对话
/quit	退出，但不删除
cwd	当前工作目录，通常就是 <code>pwd</code> 的路径

1. codex vs cx

- cx 只是 `codex --no-alt-screen` 的别名。
- 功能不变，只改显示方式。

codex = 临时全屏

cx = 保留滚动历史

两张纸比喻

正常纸 = 平时敲命令的地方

临时纸 = 给全屏程序临时借用的地方

codex 切到临时纸；cx 不换纸。

2. 为什么 split 里更适合用 cx

- Ghostty split / tmux / zellij 的 pane 更像“工作台”。
- 你通常想保留之前的命令、日志、报错和路径。
- cx 可以让这些历史和 Codex 对话留在同一份 scroll-back 里。

选择建议

工作台 pane: cx

给 Codex 独占的小全屏窗格: codex

3. cwd 是什么

cwd = current working directory = 当前工作目录

cd ~/test

pwd

/Users/daqian_mbp/test

cx

这次会话的 cwd 基本就是 `/test`。

项目管理铁律

先 cd 到项目根目录，再启动或恢复 Codex。

4. 正确的心智模型

- 项目 = 目录 / cwd
- thread 或 session = 一条保存下来的对话
- thread name = 这条对话的名字
- session ID = 这条对话的唯一 UUID
- turn = 一轮问答

不要这样想

项目

- thread
 - 多个 session

更应该这样想

项目 ~/test

- 对话 A
- 对话 B
- 对话 C

5. 三个动作的区别

codex / cx = 新建一条对话

resume = 回到旧的那条继续

fork = 从旧的那条分叉出新的一条

/quit = 退出，但不删除

关系图

cx

- 新建 对话 A

resume A

- 继续 对话 A

fork A

- 新建 对话 B

6. 为什么你的 resume 列表很长

第 1 天: cx -> 新建 A

第 2 天: cx -> 新建 B

第 3 天: codex -> 新建 C

结果：同一个项目里出现很多条独立对话。

本质

应该 resume 的场景里，反复用了新建。

7. 5 条项目管理规则

1. 总是先 `cd` 到项目根目录。
2. 一个项目尽量维护一条主线程。
3. 回到旧项目优先用 `codex resume` 或 `codex resume --last`。
4. 需要分叉实验时用 `codex fork`。
5. 长期线程用 `/rename` 命名。

推荐命名

`test: main`

`test: experiment`

`terminal-config: main`

8. 最常用命令

```
codex
cx
codex resume
codex resume --last
codex resume --all
codex resume <SESSION_ID>
codex fork <SESSION_ID>
```

贴墙总结

先 `cd` 到项目根目录

主线用 `resume`

分叉用 `fork`

长期线程用 `/rename`

`codex` = 临时全屏

`cx` = 保留滚动历史